

```
# Step 1: Load dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

INPUT = "Lifestyle_and_Health_Risk_Prediction_Synthetic_Dataset.csv"

df = pd.read_csv(INPUT)
print("Dataset shape:", df.shape)
display(df.head())
```

✓ 3.8s

Dataset shape: (5000, 12)

	age	weight	height	exercise	sleep	sugar_intake	smoking	alcohol	married	profession	bmi	health_risk
0	56	67	195	low	6.1	medium	yes	yes	yes	office_worker	17.6	high
1	69	76	170	high	6.9	high	no	no	no	teacher	26.3	high
2	46	106	153	high	6.6	low	yes	no	no	artist	45.3	high
3	32	54	186	medium	8.5	medium	no	no	no	artist	15.6	low
4	60	98	195	high	8.0	low	no	no	yes	teacher	25.8	high

Step 2 — Inspect and normalize common 'unknown' tokens

Replace common tokens that represent missing values (like `'?'`, `'NA'`, `'unknown'`) with actual `NaN` so we can impute them consistently. Then show missing value counts.

```
# Step 2: Normalize unknown tokens and inspect missing values
unknown_tokens = ["unknown", "Unknown", "UNKNOWN", "?", "NA", "N/A", "na", "n/a", "-", ""]
df.replace(unknown_tokens, np.nan, inplace=True)

print("Missing values per column (after normalizing unknown tokens):")
display(df.isna().sum())
```

✓ 0.0s

Missing values per column (after normalizing unknown tokens):

```
age          0
weight       0
height       0
exercise     0
sleep        0
sugar_intake 0
smoking      0
alcohol      0
married      0
profession   0
bmi          0
health_risk  0
dtype: int64
```

```
# Step 3: Drop duplicates and display info
before = df.shape[0]
df.drop_duplicates(inplace=True)
after = df.shape[0]
print(f"Dropped {before - after} duplicate rows. New shape: {df.shape}")
print("\nDataFrame info:")
display(df.info())
```

✓ 0.0s

Dropped 0 duplicate rows. New shape: (5000, 12)

DataFrame info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5000 entries, 0 to 4999

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	age	5000 non-null	int64
1	weight	5000 non-null	int64
2	height	5000 non-null	int64
3	exercise	5000 non-null	object
4	sleep	5000 non-null	float64
5	sugar_intake	5000 non-null	object
6	smoking	5000 non-null	object
7	alcohol	5000 non-null	object
8	married	5000 non-null	object
9	profession	5000 non-null	object
10	bmi	5000 non-null	float64
11	health_risk	5000 non-null	object

dtypes: float64(2), int64(3), object(7)

memory usage: 468.9+ KB

None

```
from sklearn.impute import SimpleImputer

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = df.select_dtypes(include=['object', 'category', 'bool']).columns.tolist()

print("Numeric columns:", num_cols)
print("Categorical columns:", cat_cols)

num_imp = SimpleImputer(strategy="median")
cat_imp = SimpleImputer(strategy="most_frequent")

if num_cols:
    df[num_cols] = num_imp.fit_transform(df[num_cols])
if cat_cols:
    df[cat_cols] = cat_imp.fit_transform(df[cat_cols])

print("\nMissing values after imputation:")
display(df.isna().sum())
display(df.head())
```

✓ 4.0s

Numeric columns: ['age', 'weight', 'height', 'sleep', 'bmi']
Categorical columns: ['exercise', 'sugar_intake', 'smoking', 'alcohol', 'married', 'profession', 'health_risk']

Missing values after imputation:

age 0
weight 0
height 0
exercise 0
sleep 0
sugar_intake 0
smoking 0
alcohol 0
married 0
profession 0
bmi 0
health_risk 0
dtype: int64

	age	weight	height	exercise	sleep	sugar_intake	smoking	alcohol	married	profession	bmi	health_risk
0	56.0	67.0	195.0	low	6.1	medium	yes	yes	yes	office_worker	17.6	high
1	69.0	76.0	170.0	high	6.9	high	no	no	no	teacher	26.3	high
2	46.0	106.0	153.0	high	6.6	low	yes	no	no	artist	45.3	high
3	32.0	54.0	186.0	medium	8.5	medium	no	no	no	artist	15.6	low
4	60.0	98.0	195.0	high	8.0	low	no	no	yes	teacher	25.8	high

```

# Step 5: IQR-based outlier detection and boxplots
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()

outlier_info = {}
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    low = Q1 - 1.5 * IQR
    high = Q3 + 1.5 * IQR
    count = ((df[col] < low) | (df[col] > high)).sum()
    outlier_info[col] = {'Q1': Q1, 'Q3': Q3, 'IQR': IQR, 'low': low, 'high': high, 'outliers': int(count)}

import pandas as pd
display(pd.DataFrame.from_dict(outlier_info, orient='index'))

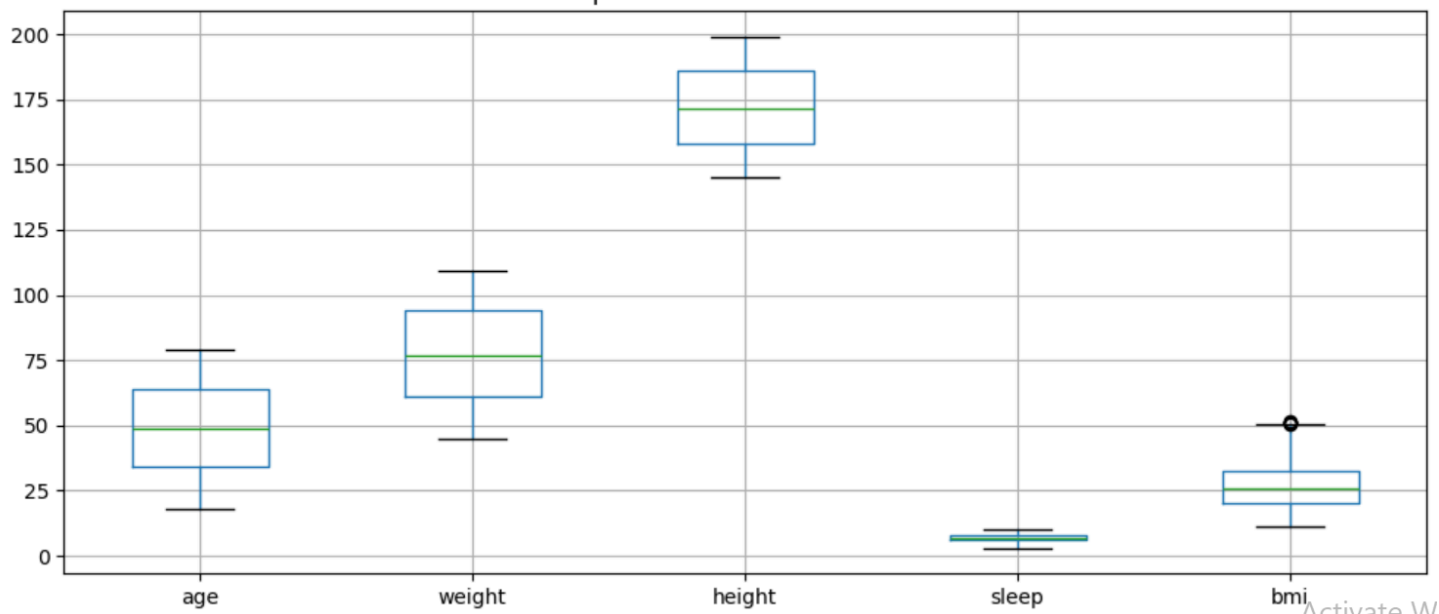
# Boxplot before winsorization
if num_cols:
    plt.figure(figsize=(12,5))
    df[num_cols].boxplot()
    plt.title("Boxplots - Before Winsorization")
    plt.show()

```

✓ 0.5s

	Q1	Q3	IQR	low	high	outliers
age	34.0	64.0	30.0	-11.00	109.00	0
weight	61.0	94.0	33.0	11.50	143.50	0
height	158.0	186.0	28.0	116.00	228.00	0
sleep	6.0	8.0	2.0	3.00	11.00	0
bmi	20.3	32.4	12.1	2.15	50.55	6

Boxplots — Before Winsorization



Activate W

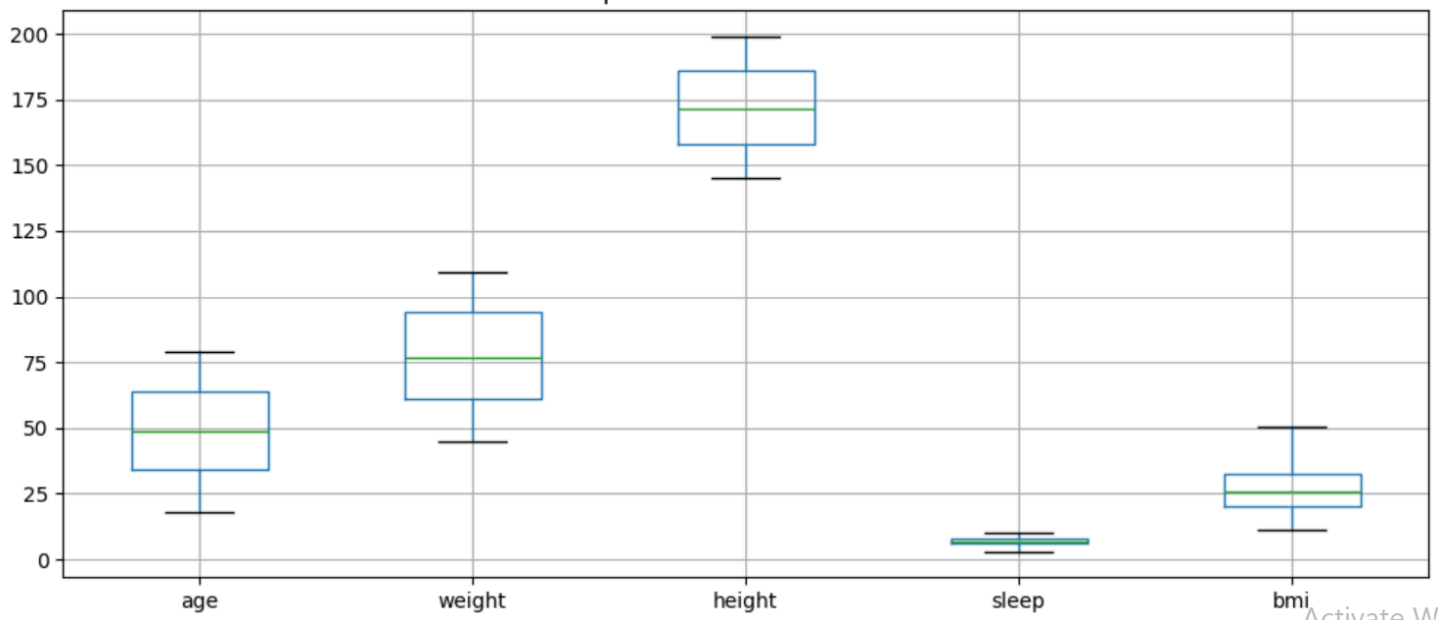
```
# Step 6: Winsorization (capping)
df_winsor = df.copy()
for col, info in outlier_info.items():
    df_winsor[col] = df_winsor[col].clip(lower=info['low'], upper=info['high'])

# Boxplot after winsorization
if num_cols:
    plt.figure(figsize=(12,5))
    df_winsor[num_cols].boxplot()
    plt.title("Boxplots – After Winsorization")
    plt.show()

display(df_winsor.head())
```

✓ 0.4s

Boxplots — After Winsorization



Activate W

	age	weight	height	exercise	sleep	sugar_intake	smoking	alcohol	married	profession	bmi	health_risk
0	56.0	67.0	195.0	low	6.1	medium	yes	yes	yes	office_worker	17.6	high
1	69.0	76.0	170.0	high	6.9	high	no	no	no	teacher	26.3	high
2	46.0	106.0	153.0	high	6.6	low	yes	no	no	artist	45.3	high
3	32.0	54.0	186.0	medium	8.5	medium	no	no	no	artist	15.6	low
4	60.0	98.0	195.0	high	8.0	low	no	no	yes	teacher	25.8	high

```
# Step 7: Feature engineering
df_fe = df_winsor.copy()

if {'height','weight'}.issubset(df_fe.columns):
    # convert height to meters if necessary
    h = pd.to_numeric(df_fe['height'], errors='coerce')
    h_m = np.where(h > 3, h / 100.0, h)
    # avoid division by zero
    h_m = np.where(h_m <= 0, np.nan, h_m)
    df_fe['BMI'] = pd.to_numeric(df_fe['weight'], errors='coerce') / (h_m ** 2)
    print("BMI feature created.")

# interaction features among top 3 numeric columns by variance (excluding target)
num_for_interact = [c for c in num_cols if c != 'target']
top3 = df_fe[num_for_interact].var().sort_values(ascending=False).index[:3].tolist()
print("Top numeric columns chosen for interactions:", top3)
for i in range(len(top3)):
    for j in range(i+1, len(top3)):
        new_col = f"{top3[i]}_x_{top3[j]}"
        df_fe[new_col] = df_fe[top3[i]] * df_fe[top3[j]]

display(df_fe.head())
```

✓ 0.0s

BMI feature created.

Top numeric columns chosen for interactions: ['weight', 'age', 'height']

	age	weight	height	exercise	sleep	sugar_intake	smoking	alcohol	married	profession	bmi	health_risk	BMI	weight_x_age	weight_x_height	age_x_height
0	56.0	67.0	195.0	low	6.1	medium	yes	yes	yes	office_worker	17.6	high	17.619987	3752.0	13065.0	10920.0
1	69.0	76.0	170.0	high	6.9	high	no	no	no	teacher	26.3	high	26.297578	5244.0	12920.0	11730.0
2	46.0	106.0	153.0	high	6.6	low	yes	no	no	artist	45.3	high	45.281729	4876.0	16218.0	7038.0
3	32.0	54.0	186.0	medium	8.5	medium	no	no	no	artist	15.6	low	15.608741	1728.0	10044.0	5952.0
4	60.0	98.0	195.0	high	8.0	low	no	no	yes	teacher	25.8	high	25.772518	5880.0	19110.0	11700.0

Activate
Go to Set

```
# Step 8: Automatically detect or assume the target column
possible_targets = ['target', 'Target', 'TARGET', 'label', 'Label', 'class', 'Class', 'outcome', 'Outcome', 'disease', 'Disease', 'risk', 'Risk']

target_col = None
for col in df_fe.columns:
    if col in possible_targets:
        target_col = col
        break

# If still not found, assume the last column is the target
if target_col is None:
    target_col = df_fe.columns[-1]
    print(f"No common target name found. Assuming the last column ({target_col}) is the target.")

print(f"Detected target column: {target_col}")

y = df_fe[target_col]
X = df_fe.drop(columns=[target_col])

print("X shape:", X.shape)
print("y distribution:")
display(y.value_counts())
```

✓ 0.0s

```
No common target name found. Assuming the last column ('age_x_height') is the target.  
Detected target column: age_x_height  
X shape: (5000, 15)  
y distribution:
```

```
age_x_height
```

```
9360.0      9
```

```
10416.0     9
```

```
7568.0      9
```

```
8910.0      9
```

```
8640.0      9
```

```
..
```

```
3920.0      1
```

```
7904.0      1
```

```
3843.0      1
```

```
12537.0     1
```

```
6408.0      1
```

```
Name: count, Length: 2208, dtype: int64
```

Numeric features: ['age', 'weight', 'height', 'sleep', 'bmi', 'BMI', 'weight_x_age', 'weight_x_height']
Categorical features: ['exercise', 'sugar_intake', 'smoking', 'alcohol', 'married', 'profession', 'health_risk']

	age	weight	height	sleep	bmi	BMI	weight_x_age	weight_x_height	exercise_low	exercise_medium	...	alcohol_yes	married_yes	profession_doctor	profession_driver	profession_eng
0	0.401805	-0.553592	1.458019	-0.626130	-1.120386	-1.117872	-0.015839	-0.068361	1.0	0.0	...	1.0	1.0	0.0	0.0	
1	1.127851	-0.072635	-0.122195	-0.067746	-0.065861	-0.066283	0.859838	-0.110345	0.0	0.0	...	0.0	0.0	0.0	0.0	
2	-0.156692	1.530554	-1.196740	-0.277140	2.237127	2.234299	0.643854	0.844573	0.0	0.0	...	0.0	0.0	0.0	0.0	
3	-0.938588	-1.248308	0.889142	1.049022	-1.362806	-1.361603	-1.203754	-0.943075	0.0	1.0	...	0.0	0.0	0.0	0.0	
4	0.625204	1.103037	1.458019	0.700032	-0.126466	-0.129913	1.233116	1.681936	0.0	0.0	...	0.0	1.0	0.0	0.0	

5 rows x 24 columns

```

from sklearn.utils import resample
try:
    from imblearn.over_sampling import SMOTE
    sm = SMOTE(random_state=42)
    X_bal, y_bal = sm.fit_resample(X_pre_df, y)
    print("SMOTE used to balance the classes.")
except Exception as e:
    print("SMOTE not available; using sklearn resample for upsampling minority.")
    df_temp = X_pre_df.copy()
    df_temp['target'] = y.values
    majority = df_temp[df_temp['target'] == df_temp['target'].mode()[0]]
    minority = df_temp[df_temp['target'] != df_temp['target'].mode()[0]]
    if minority.shape[0] == 0:
        X_bal = X_pre_df.values
        y_bal = y.values
        print("No minority class detected; skipping balancing.")
    else:
        minority_up = resample(minority, replace=True, n_samples=majority.shape[0], random_state=42)
        df_bal = pd.concat([majority, minority_up])
        X_bal = df_bal.drop(columns=['target']).values
        y_bal = df_bal['target'].values
        print("Upsampled minority class with sklearn resample.")

import numpy as np
print("Balanced shapes:", np.shape(X_bal), np.shape(y_bal))

```

✓ 0.0s

```

SMOTE not available; using sklearn resample for upsampling minority.
Upsampled minority class with sklearn resample.
Balanced shapes: (18, 24) (18,)

```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_bal)

pca = PCA(n_components=0.95, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print("Original feature count:", X_scaled.shape[1])
print("Reduced feature count (PCA):", X_pca.shape[1])
print("Explained variance sum:", pca.explained_variance_ratio_.sum())
```

✓ 0.0s

```
Original feature count: 24
Reduced feature count (PCA): 11
Explained variance sum: 0.9671524389066766
```



```
import matplotlib.pyplot as plt
import numpy as np

# Ensure y_bal is a numpy array aligned with X_pca
y_plot = np.array(y_bal)

plt.figure(figsize=(8,6))
if X_pca.shape[1] >= 2:
    plt.scatter(X_pca[:,0], X_pca[:,1], c=y_plot, cmap='viridis', alpha=0.6)
    plt.xlabel('PC1')
    plt.ylabel('PC2')
    plt.title('PCA Scatter Plot (PC1 vs PC2) colored by target')
    plt.colorbar(label='target')
    plt.show()
else:
    print("PCA produced fewer than 2 components; scatter plot not available.")
```

✓ 0.1s

